

Proyecto:	DungeonScroll
Alumno:	Alexandre Barbeito Domínguez
Grupo:	DAM2 curso 2025-2026

Centro educativo

Código	Centro	Concello	Ano académico
15005397	I.E.S. Fernando Wirtz Suárez	A Coruña	2025 - 2026

Ciclo formativo

Código da familia profesional	Familia profesional	Código do ciclo formativo	Ciclo formativo	Grao	Réxime
FP16	Informática e comunicacións	CSINF02	Desenvolvemento Aplicacións Multiplataforma.	Superior	Dual

Módulo profesional e unidades formativas de menor duración (*)

Código MP/UF	Nome
MP0492	Proxecto de Desenvolvemento Aplicacións Multiplataforma. Equivalencia en créditos ECTS: 5. Código: MP0492. Duración: 26 horas

Profesorado responsable

Titora	Gomez Mosquera, Marian
Equipo Docente:	Grela Castro, Jesús Dopazo, Rodrigo Pazos, José Manuel Vila Muñoz, Noe

Alumno

Alumno/a	Alexandre Barbeito Domínguez
-----------------	------------------------------

Datos do Proxecto

Título	DungeonScroll: Gestor de Personajes D&D 5e
---------------	--

Índice:

1. Objetivo.....	2
2. Descripción.....	3
3. Alcance	3
4. Planificación	4
5. Medios a utilizar	5
6. Presupuesto	6
7. Ejecución	7
8. Diseño de la Base de Datos (Modelo ER)	7
9. Integración de Datos y Marco Legal.....	13
10. Cumplimiento de Requisitos por Módulo (Estado Actual)	13

1. Objetivo

1.1) Objetivo General

Desarrollar una aplicación multiplataforma que centralice la lógica de negocio de D&D 5e, permitiendo la persistencia de datos, la automatización de cálculos de reglas y la gestión de personajes mediante una interfaz moderna y reactiva.

1.2) Objetivos Específicos

- Normalización de Reglas: Diseñar una base de datos de 37 entidades que cubra la totalidad del sistema de juego.
- Sincronización Inteligente: Consumir y almacenar datos de APIs externas para asegurar el funcionamiento sin depender de ellas.
- Lógica de Progresión: Automatizar el sistema de subida de nivel, incluyendo la gestión de dotes (feats), dotes de clase y recursos específicos.
- Control de Acceso: Implementar un sistema de gestión de usuarios privado, donde el acceso es controlado por administración.

2. Descripción

DungeonScroll es un sistema digital diseñado para la creación y gestión de personajes de Dungeons & Dragons 5º Edición. El proyecto nace de la necesidad de automatizar las complejas reglas del sistema (cálculos de estadísticas, gestión de niveles, inventario y magia), ofreciendo una experiencia fluida para dispositivos móviles.

El sistema se basa en una arquitectura cliente-servidor robusta:

- **Backend:** API REST desarrollada en Java 21 con Spring Boot.
- **Frontend:** Aplicación móvil multiplataforma desarrollada con Flutter.
- **Base de Datos:** Relacional MySQL en contenedores mediante Docker.
- **Despliegue:** Hosting en servidor remoto (VPS) para disponibilidad 24/7 y acceso desde móviles.

3. Alcance

Funcionalidades Principales:

- **Gestión de Personajes:** Creación guiada, cálculo automático de modificadores, bonificadores de competencia y puntos de golpe.
- **Lógica de Progresión:** Automatización de subida de nivel, incluyendo dotes de clase y recursos específicos.
- **Sincronización:** Consumo de la API externa *dnd5eapi.co* con un sistema de *ApiRateLimiter* para evitar bloqueos y asegurar copia local.
- **Seguridad:** Sistema de gestión de usuarios cerrado (controlado por admin) para proteger la integridad del servidor.

Fuera de Alcance:

- Integración con VTT (Roll20, Foundry).
- Herramientas exclusivas para el Dungeon Master (gestión de combates o wikis de campaña).

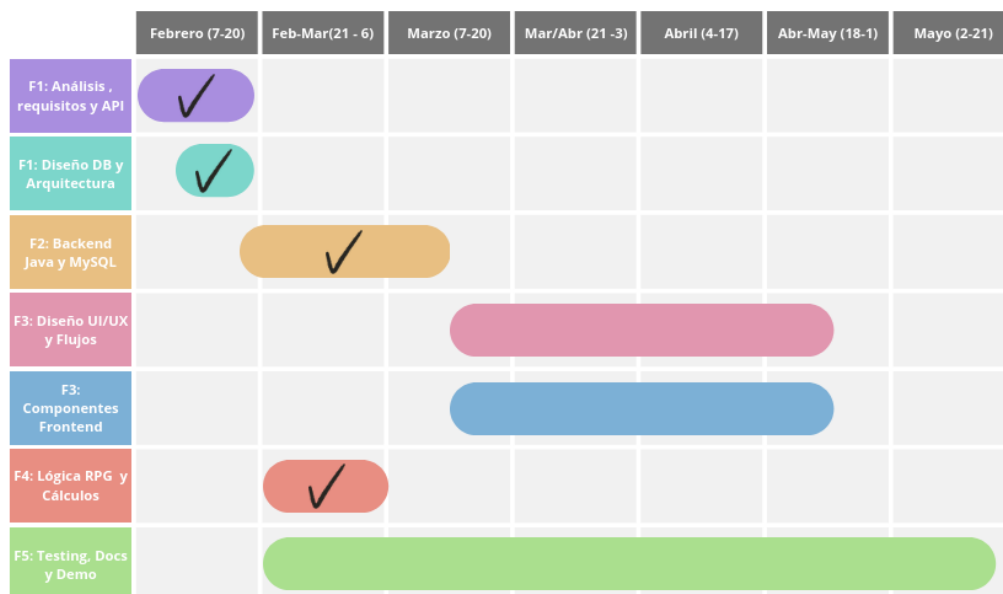
4. Planificación

Tabla de Seguimiento de Fases

Fase	Tarea	Estado	Observaciones
1	Análisis y Diseño	100%	Finalizado según planificación inicial
2	Backend y Persistencia	100%	Incluye CRUDs, Sincronización y Docker
3	Frontend (UI/UX)	5%	Pantalla de Login y Dashboard básico
4	Lógica Avanzada	100%	Implementada en Backend (Cálculos, Level-Up, etc)
5	Testing y Entrega	10%	Debuggeo constante. Readme actualizado al día.

El tiempo planificado para cada fase fue respetado en su mayoría. La fase 1 se realizó en la fecha prevista. La fase 2 se completó antes de lo esperado, aunque esto provocó un retraso en la fase 3, ya que se consideró preferible finalizar correctamente el backend antes de comenzar con el desarrollo del frontend.

En cuanto a la fase 4, durante el proceso de la fase 2 se hizo necesario desarrollar la lógica avanzada al mismo tiempo que el backend, ya que formaba parte del mismo, quedando así completada. Respecto a la fase 5, aún queda bastante trabajo por realizar en lo referente al testeo y la documentación. Aquí un diagrama actualizado con la nueva planificación:



5. Medios a utilizar

Hardware:

- Ordenador personal para desarrollo.
- Conexión a internet (necesaria para exportar datos desde la API).
- Servidor Virtual Privado (VPS).

Arquitectura y tecnologías:

- **Backend** (Java Spring Boot 3.20)
 - o Modelo: MVC con servicios individuales para cada entidad.
 - o Persistencia: Spring Data JPA con Hibernate para gestionar relaciones complejas (Many-to-Many, Herencia de entidades)
 - o Gestión de carga: Implementación de ApiRateLimiter para la sincronización con servidores externos, respetando límites de peticiones.
- **Frontend** (Flutter y Dart)
 - o Patrón: MVVM (Model-View-ViewModel) mediante el uso de Provider.
 - o Interfaz: Material Design 3, optimizado para la visualización de fichas de personaje con información densa.
- **Infraestructura**
 - o Docker: Se ha decidido utilizar Docker Compose para la gestión de la base de datos MySQL, garantizando que el entorno de desarrollo sea idéntico al de producción y facilitando la portabilidad.
 - o Despliegue y Hosting: El sistema backend y la base de datos se encuentran dockerizados y desplegados en un Servidor Virtual Privado (VPS). Se utiliza Docker Compose para organizar los contenedores en la nube, permitiendo que la aplicación móvil se conecte de forma remota a la API REST, asegurando que el servicio esté operativo de forma ininterrumpida.

- Herramientas de desarrollo:

- o IntelliJ Idea (backend)
- o Visual Studio Code (Flutter)
- o DBeaver (gestión de BD)
- o Control de versiones: Github
- o **API externa: dnd5api.co**

- Recursos

- o Documentación oficial de Flutter, Spring Boot y MySQL.
- o Apuntes recibidos durante el curso.
- o Documentación de la API utilizada.
- o Documentación sobre D&D en foros de internet, manuales impresos/PDF y webs de wiki.

6. Presupuesto

Se mantiene la estimación inicial de mano de obra y licencias, añadiendo el coste de la infraestructura del servidor.

Concepto	Detalle	Total (est.)
Mano de Obra	26h x 40,00 €/h	1040,00 €
Licencias Software	IntelliJ, Suscripción IA, D&D Beyond	236,00 €
Infraestructura	Hosting VPS (4 meses aprox)	60,00 €
Subtotal		1336,00 €
IVA (21%)		280,56 €
TOTAL		1.616,56 €

7. Ejecución

La ejecución final consistiría en una demostración funcional donde se muestre:

1. Inicio de la aplicación y acceso a lista de personajes.
2. Creación de un nuevo personaje seleccionando raza, clase y demás opciones con ayuda de un wizard.
3. Introducción de características y cálculo automático de modificadores.
4. Guardado del personaje en base de datos en servidor.
5. Visualización de ficha digital con estadísticas completas en diferentes pestañas.
6. Subida de nivel y actualización automática de valores.

8. Diseño de la Base de Datos (Modelo ER)

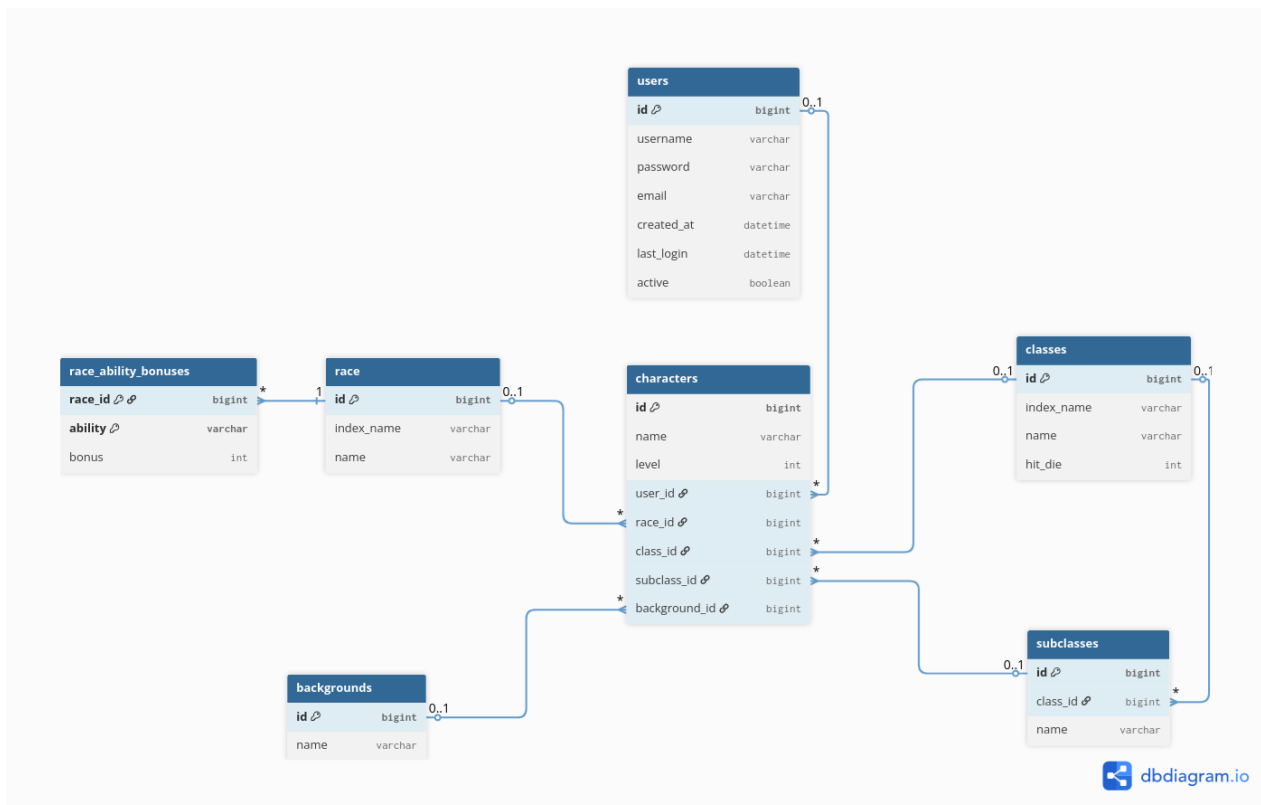
La base de datos es el núcleo de DungeonScroll. Actualmente cuenta con 37 entidades principales, organizadas en los siguientes bloques lógicos:

- **Núcleo:** *PlayerCharacter, DndClass, Race, Background.*
- **Habilidades y Combate:** *Skill, CharacterSkill, SavingThrow, Condition, DamageType.*
- **Sistema de Magia:** *Spell, CharacterSpell, SpellSlotProgression.*
- **Inventario y Economía:** *Item, CharacterInventory, CharacterEquipment, CharacterMoney.*
- **Progresión de Clase:** *ClassFeature, Subclass, LevelUpTask, ClassResource.*

Dado que un esquema completo de la base de datos resultaría visualmente inmanejable, se ha segmentado en cinco diagramas parciales para una mejor explicación.

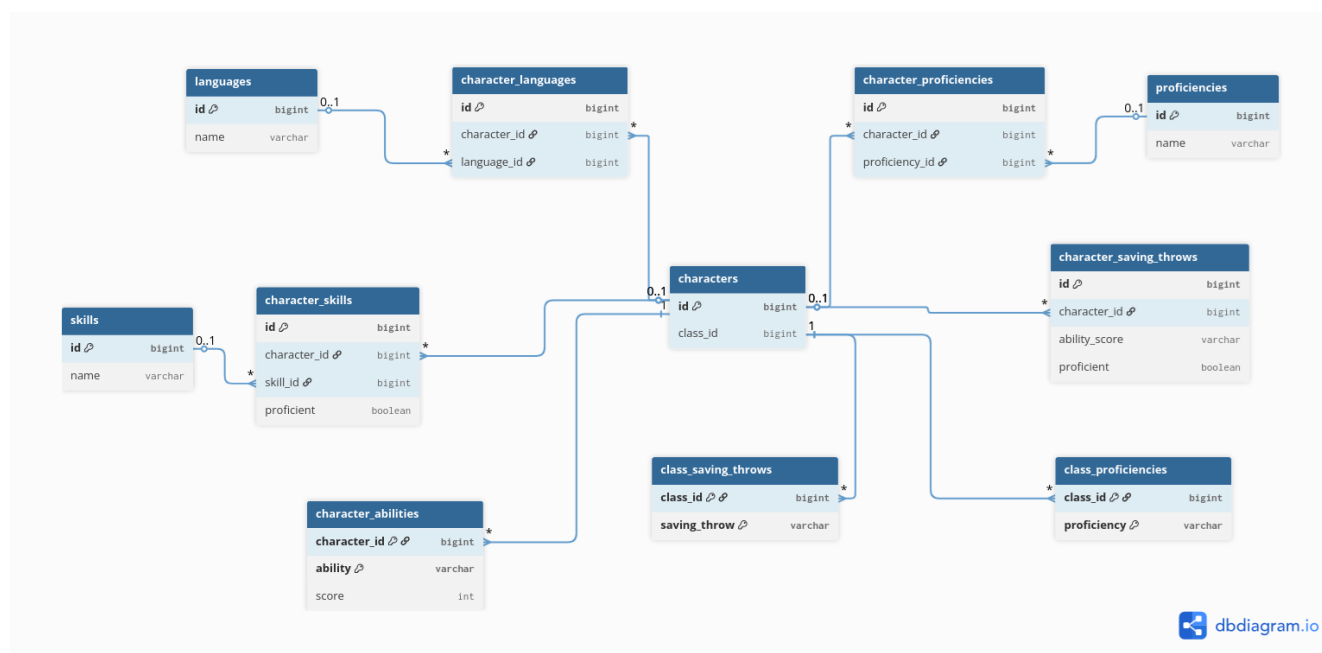
1. Núcleo y Usuarios (Estructura Base)

Representa la columna vertebral del sistema. Se gestiona la identidad del **usuario** y cómo se vincula con el **personaje**. Cada **usuario** tiene acceso a una lista de **personajes**, que al crearlo deberá escoger entre su **raza**, **clase**, su **subclase** (que depende a su vez de su clase), trasfondo y una serie de datos que no vienen recogidos en este diagrama de muestra, ya que la tabla **personaje** se relaciona prácticamente con todas las demás tablas al ser el eje principal.



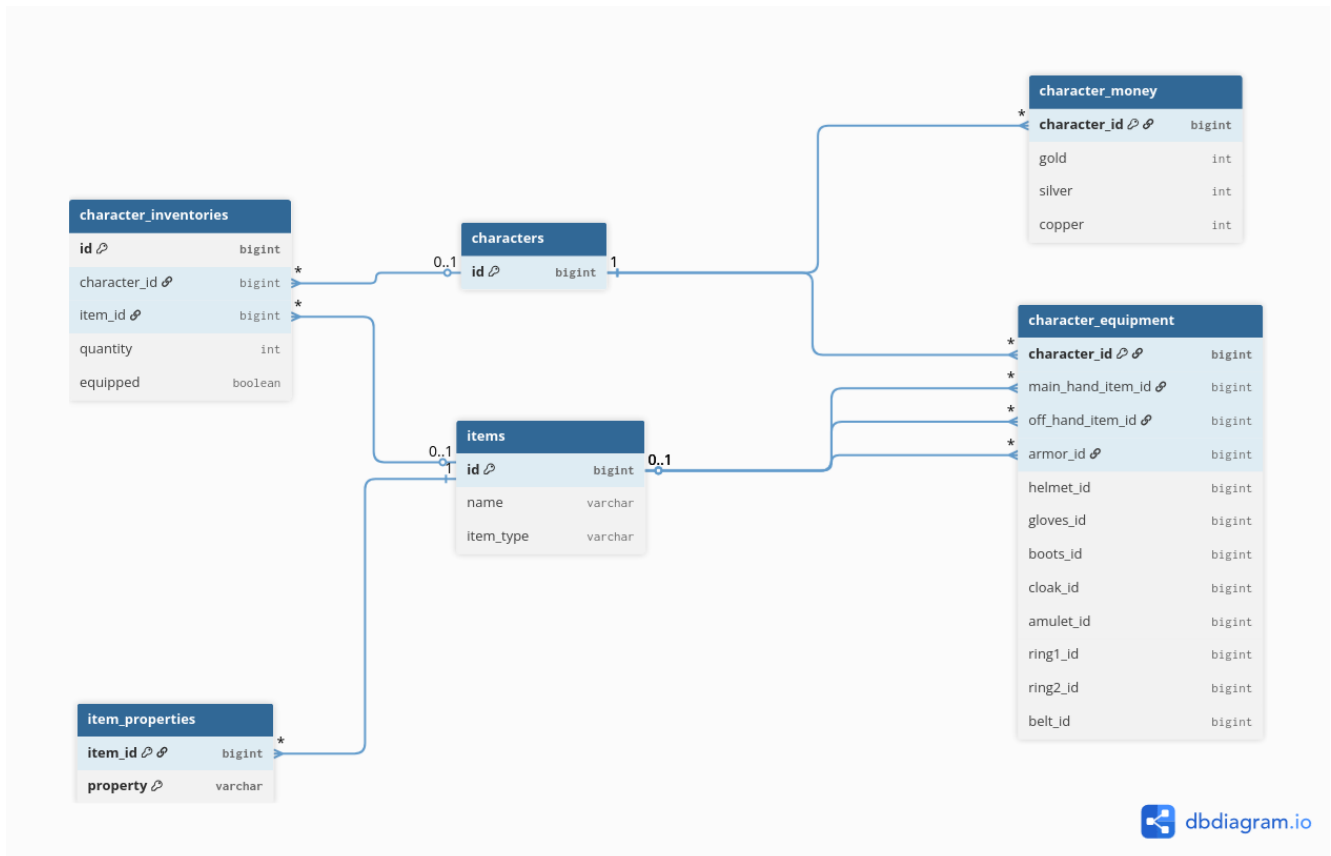
2. Capacidades y Atributos

En este módulo se expone lo que el **personaje** “es” y “sabe hacer”. En este caso la relación de **personajes** con el resto de tablas es de *1 a muchos* ya que cada personaje puede tener asociado 1 o más **idiomas, características, habilidades, salvaciones, competencias**, etc. Tablas como **skills** o **languages** vienen separadas para permitir que sea fácilmente extensible con contenido de expansiones (homebrew o suplementos oficiales)



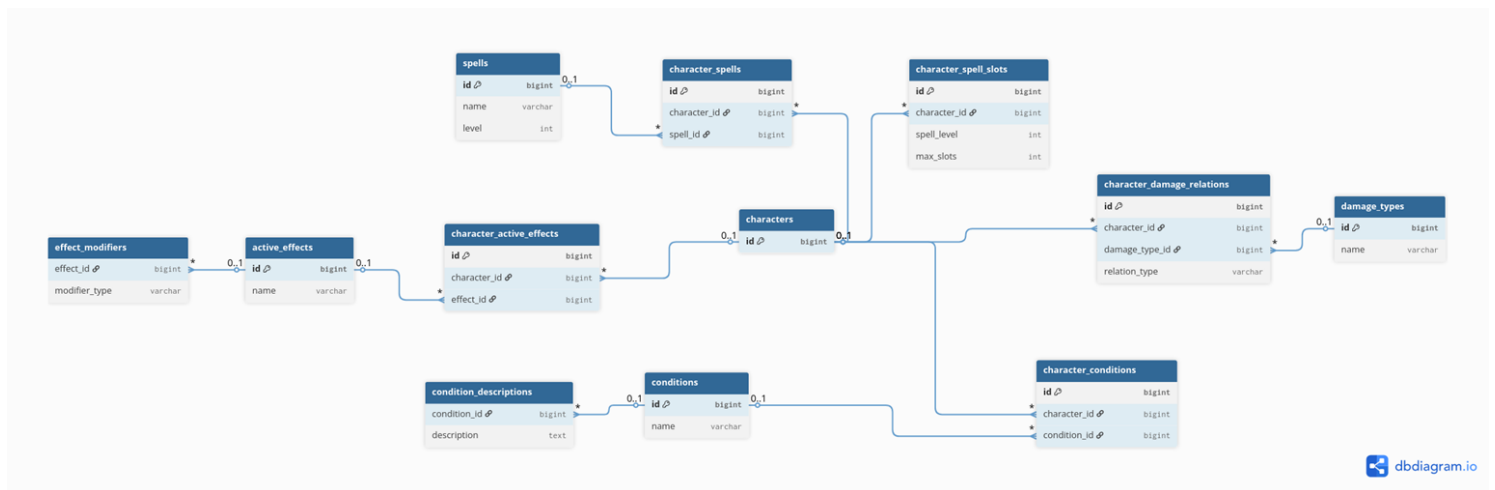
3. Inventario y Equipo

Aquí se gestiona la posesión física y el estado de objetos. Separamos **character_inventories** (qué tienes y cuánto) y **character_equipment** (qué tienes puesto) para poder automatizar cálculos como la Clase de Armadura (CA) o el peso transportado. Se ha separado en "slots" específicos (casco, peto, anillos...) para poder gestionar mejor la cantidad de unidades de cada tipo de item que un personaje puede llevar puesto.



4. Magia y Efectos

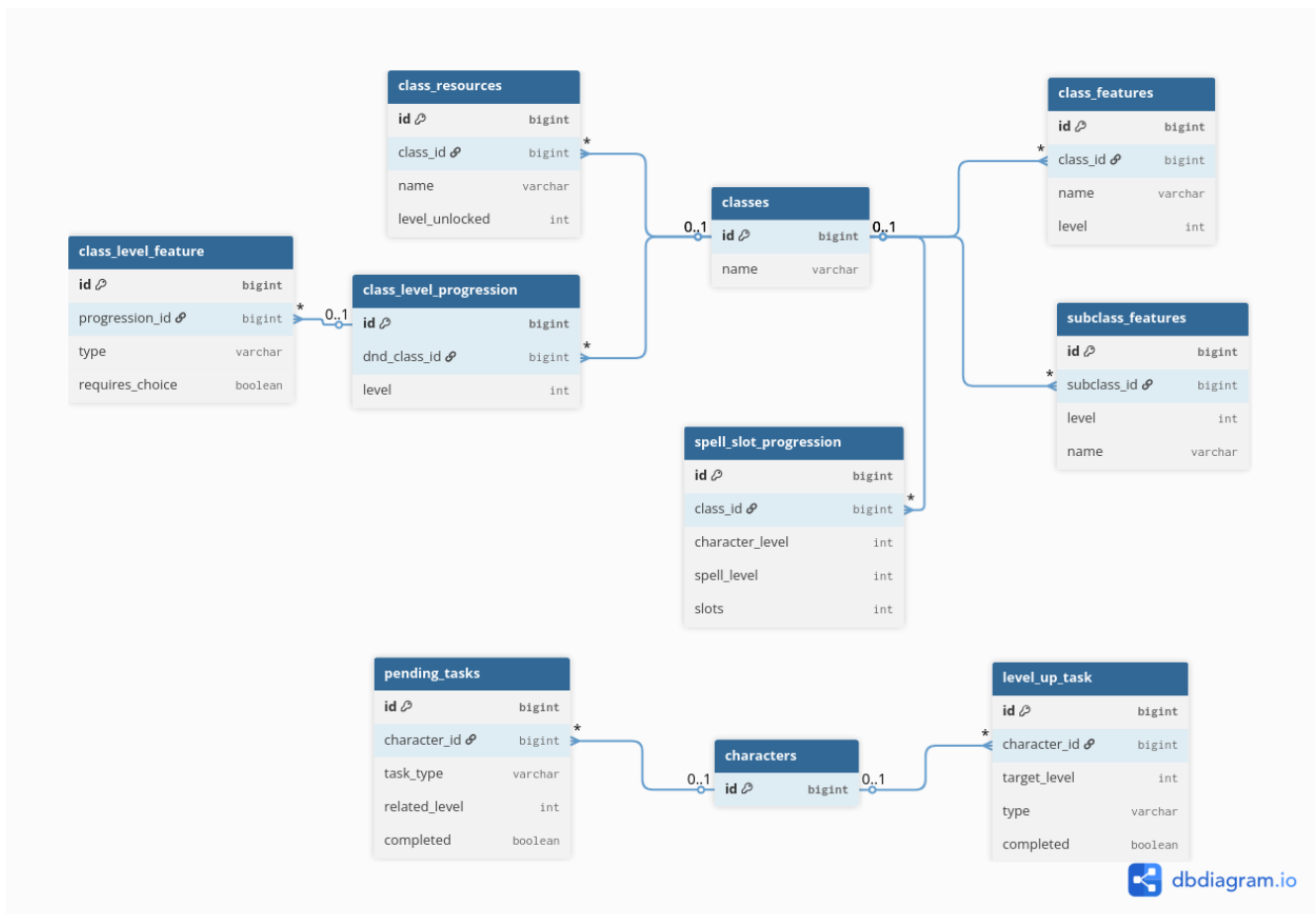
Aquí se gestiona desde la preparación de hechizos y el gasto de espacios (**spell_slots**), hasta las condiciones (envenenado, derribado, etc) y los efectos activos de larga duración. Se diseñó de forma relacional para que el sistema pueda "limpiar" automáticamente efectos al terminar su duración o permitir aplicar modificadores de daño basados en las **damage_relations** (resistencias o vulnerabilidades) del personaje en tiempo real.



5. Progresión y Clases

Este diagrama representa el motor de **subida de nivel**. Las tablas representan los rasgos que se consiguen cuando se sube de nivel, adquiriendo nuevas dotes otorgadas por su clase, su subclase, así como qué recursos limitados se obtienen específicamente de algunas clases (ej: puntos de Ki, Furia, Espacios de Conjuro...). Separación en 2 tablas como es en el caso de **class_features** y **class_level_feature** o entre **pending_tasks** y **level_up_task** es necesaria ya que unas están inherentemente relacionadas a la subida de nivel.

Es una arquitectura basada en eventos que asegura que no se pierdan elecciones importantes (como escoger un hechizo nuevo o un talento) durante el proceso de crecimiento de un personaje.



9. Integración de Datos y Marco Legal

Uno de los retos del proyecto es la gestión de la propiedad intelectual de Wizards of The Coast (WoTC).

- **Datos Oficiales (SRD):** La aplicación consume la API pública dnd5eapi.co, que contiene el contenido bajo la licencia Open Game License (OGL).

- **Expansiones y Contenido Personalizado:** Para uso personal y de testeo, se ha explorado la integración de los repositorios de contenido de expansión o incluso "homebrew".

- **Aviso Legal:** Soy consciente de que el contenido de libros de expansión (non-SRD) está protegido por copyright. Por ello, la aplicación se plantea como una herramienta de gestión de datos dirigida a usuarios, pero no como un producto comercial. El sistema de creación de usuarios es cerrado (gestionado por administrador) para evitar un uso público indebido de materiales protegidos.

10. Cumplimiento de Requisitos por Módulo (Estado Actual)

En esta sección se detalla cómo el proyecto aborda todas las competencias específicas requeridas.

Programación Multimedia y Acceso a Datos

- **Persistencia en el dispositivo:**

- Estado: En implementación.
- Detalle: Se está utilizando el almacenamiento seguro del dispositivo para la persistencia del token JWT y preferencias básicas. Para la fase final de la aplicación móvil, se evaluará el uso de persistencia local de personajes para permitir el modo offline, asegurando la integridad de los datos entre el servidor y el cliente.

- **Persistencia en Servidor:** La base de datos MySQL reside en un volumen persistente dentro del VPS, garantizando que

los datos de los personajes creados por el usuario se mantengan seguros y disponibles.

- **Consumo de APIs:**

- Estado: Cumplido al 100% (Backend) / En desarrollo (Frontend).
- Detalle: El sistema realiza un consumo intensivo de la API externa dnd5api.co. Se procesan respuestas JSON complejas para mapear más de 300 hechizos, 12 clases y sus progresiones automáticas hacia la base de datos MySQL.

Programación de Servicios y Procesos

El proyecto se ha diseñado como un sistema que integra ambas alternativas propuestas, realizando un consumo de servicios externos y de servicios propios.

- **Consumo de API externa (Opción 1):**

- Estado: Cumplido al 100%.
- Detalle: El backend actúa como cliente de la API pública dnd5api.co.
- Operaciones: Se realizan múltiples operaciones de consulta y filtrado (clases, hechizos, razas, dotes, etc) y envío de datos procesados hacia la base de datos local.
- Gestión de datos: Los datos se obtienen en formato JSON, se procesan mediante RestTemplate y se integran en el modelo relacional propio.

- **Creación de API Propia (Opción 2):**

- Estado: Cumplido al 100%.
- Detalle: Se ha desarrollado una API REST propia con más de 25 controladores funcionales.
- Endpoints: Implementación completa de operaciones CRUD para personajes, inventarios, hechizos y dinero (Ej: GET /api/characters, POST /api/characters/{id}/level-up).

- Accesibilidad en Red: La API ha sido publicada en un servidor remoto (VPS), configurando reglas de firewall y gestión de puertos para permitir la conexión del usuario desde el dispositivo móvil desde cualquier red externa.
- Pruebas: Todos los endpoints han sido validados mediante Postman y documentados en el Readme técnico.
- **Concurrencia y Paralelismo:**
 - Estado: Cumplido.
 - Detalle:
 - Backend: El servidor Spring Boot gestiona un pool de hilos para atender peticiones simultáneas. Se ha implementado un ApiRateLimiter para gestionar la sincronización con la API externa de forma asíncrona, evitando el bloqueo del sistema durante la carga masiva de datos.
 - Frontend: Uso de tareas asíncronas (Future y async/await) en Flutter para las llamadas al API, garantizando que la interfaz de usuario no se congele durante la comunicación en red.

Desarrollo de Interfaces

- **Diseño y Estilo**: Aplicación de Material Design 3, con una paleta de colores coherente y componentes reutilizables (Tarjetas de personaje, botones de acción, etc).
- **Testing**: Pendiente de ejecución formal. Se han realizado pruebas de usabilidad manuales en el flujo de login y navegación básica del dashboard.

Empresa e Iniciativa Emprendedora

- **Idea de Negocio y Valor**: Herramienta de nicho para jugadores de rol que buscan automatizar la hoja de personaje sin depender de suscripciones costosas.

- **Aspectos Legales y el SRD:** El proyecto se ciñe al SRD (System Reference Document) de Wizards of The Coast.
- **Nota:** Para uso privado con amigos, se ha previsto la integración de datos del repositorio Aurora Legacy, con contenido que no pertenece al SRD. Se hace constar que, debido a derechos de autor, la aplicación se plantea como una herramienta de gestión privada y no como un producto comercializable fuera del marco de la Fan Content Policy.
- **Modelo de Usuarios:** No se implementará registro público. Un administrador gestionará las cuentas para mantener el control sobre la carga del servidor y el uso del contenido.

Inglés Profesional

- **Abstract:** Pendiente de redacción final. Se incluirá un resumen ejecutivo con la terminología técnica adecuada (JPA, REST API, Provider, State Management).
- **Presentación:** Se preparará siguiendo la estructura de Greetings, Introduction, Timeline, Product Test and Conclusions.